

Flash-SD-KDE: Accelerating SD-KDE with Tensor Cores

Anonymous Authors¹

Abstract

Score-debiased kernel density estimation (SD-KDE) achieves improved asymptotic convergence rates over classical KDE, but its use of an empirical score has made it significantly slower in practice. We show that by re-ordering the SD-KDE computation to expose matrix-multiplication structure, Tensor Cores can be used to accelerate the GPU implementation. On a 32k-sample 16-dimensional problem, our approach runs up to $40\times$ faster than a strong SD-KDE GPU baseline and $2,800\times$ faster than scikit-learn’s KDE. On a larger 1M-sample 16-dimensional task evaluated on 131k queries, SD-KDE completes in 2.4s on a single GPU, making score-debiased density estimation practical at previously infeasible scales. Code available in supplementary material.

1. Introduction

Score-debiased kernel density estimation (SD-KDE) (Epstein et al., 2025) improves the statistical efficiency of standard kernel density estimators: for sufficiently smooth densities, SD-KDE attains better bias and mean-squared error rates than vanilla KDE while retaining a simple, nonparametric form. In particular, SD-KDE achieves an asymptotic mean integrated squared error (AMISE) of order $O(n^{-8/(d+8)})$, improving the convergence exponent relative to the $O(n^{-4/(d+4)})$ rate of classical KDE tuned with Silverman’s rule of thumb. These accuracy gains come with a significant computational drawback: the empirical score used for debiasing introduces an additional $O(n^2)$ pass over the data, so a naive implementation has roughly the same quadratic cost as KDE itself but with a larger constant factor.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Given samples x_i and a bandwidth h , a Gaussian KDE is

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^n \varphi\left(\frac{x - x_i}{h}\right),$$

and SD-KDE forms debiased samples $x_i^{\text{SD}} = x_i + \frac{h^2}{2} \hat{s}(x_i)$ where $\hat{s}$ is an estimate of the score. In this work we always use an empirical score computed from the KDE itself, i.e. no parametric model for the underlying density is assumed. Writing the KDE explicitly in terms of the samples,

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^n \varphi\left(\frac{x - x_i}{h}\right),$$

we estimate the score as

$$\hat{s}(x) = \frac{\partial_x \hat{p}(x)}{\hat{p}(x)} = \frac{\sum_{i=1}^n \left[-\frac{x - x_i}{h} \varphi\left(\frac{x - x_i}{h}\right) \right]}{h^2 \sum_{i=1}^n \varphi\left(\frac{x - x_i}{h}\right)}.$$

2. Hardware

All experiments run on a workstation equipped with an NVIDIA RTX A6000 (GA102). This GPU exposes 84 streaming multiprocessors (SMs); each SM contains 128 FP32 ALUs and 16 special function units (SFUs). Because the ratio of FP32 ALUs to SFUs is 128:16, we treat one exp (issued on an SFU) as costing the equivalent of $128/16 = 8$ FP32 flops in our models. The card delivers roughly 40 TFLOP/s of peak FP32 throughput and approximately 770 GB/s of GDDR6 bandwidth.

The host CPU is a dual-socket AMD EPYC 7763 system (“Milan”) configured with 2×64 cores and two hardware threads per core (256 logical CPUs reported by `lscpu`). The processors boost up to 3.53 GHz, expose 512 MiB of shared L3 cache across the sockets, and provide modern ISA extensions (AVX/AVX2, FMA, BMI1/2, SHA, VAES, etc.).

3. Method

Our goal is to take n_{train} samples in d dimensions, form the empirical SD-KDE (score + shift), and evaluate the resulting density on n_{test} queries. Writing the computation in matrix form highlights the parallel structure. Let $x_i, y_j \in \mathbb{R}^d$

denote training and query points, with bandwidth h and squared Euclidean distance

$$\|x_i - y_j\|^2 = \|x_i\|^2 + \|y_j\|^2 - 2x_i^\top y_j.$$

Stacking training samples into $X \in \mathbb{R}^{n_{\text{train}} \times d}$ and queries into $Y \in \mathbb{R}^{n_{\text{test}} \times d}$, the pairwise dot-product matrix

$$G = XY^\top \in \mathbb{R}^{n_{\text{train}} \times n_{\text{test}}}$$

dominates the arithmetic once d is moderately large ($d \gtrsim 16$). On modern NVIDIA GPUs this GEMM can be mapped to Tensor Cores and evaluated at 5–10 $\times$ the throughput of standard FP32 SIMT arithmetic, particularly when we restrict to d that are multiples of 16 and use Triton’s `tl.dot` interface. The remaining operations—vector norms, broadcasted additions, and exponentials—are all $O(n_{\text{train}}n_{\text{test}})$ but quickly become a small fraction of the total FLOPs as d grows.

The empirical SD-KDE score inherits the same GEMM structure. Its numerator involves terms of the form

$$\sum_j -(x_i - y_j) \varphi_{ij}, \quad \varphi_{ij} = \exp\left(-\frac{\|x_i - y_j\|^2}{2h^2}\right),$$

which naively suggests $O(n_{\text{train}}n_{\text{test}}d)$ additional element-wise arithmetic. Using the identity

$$\sum_j (x_i - y_j) \varphi_{ij} = x_i \sum_j \varphi_{ij} - \sum_j \varphi_{ij} y_j,$$

we can decompose the numerator into a second GEMM:

$$T = \Phi Y.$$

Thus both the KDE evaluation and the SD-KDE score numerator reduce to Tensor-Core-accelerated matrix multiplies, plus $O(n^2)$ scalar work for the norms and exponentials. In this note we focus on the $d = 16$ case, which aligns naturally with the Tensor Core tile sizes on the RTX A6000.

3.1. Arithmetic intensity in d dimensions

To understand when the high-dimensional SD-KDE becomes compute-bound, we estimate FLOPs, bytes moved, and arithmetic intensity for the d -dimensional case. Let $n_{\text{train}} = k$ and set $n_{\text{test}} = k/8$ as in the experiments.

Total FLOPs. The 16-D implementation consists of three main matrix-multiply stages:

1. Score Gram matrix $G = XX^\top$: $2dk^2$ FLOPs.
2. Score numerator $T = \Phi X$: $2dk^2$ FLOPs, plus $4k^2$ scalar FLOPs for norms and distance terms and $8k^2$ FLOPs for exponentials (counting each exp as 8 FLOPs due to the SFU/FP32 ratio).

3. Final KDE Gram matrix on debiased data: $2dk(k/8)$ FLOPs, plus $4k(k/8)$ scalar FLOPs and $8k(k/8)$ FLOPs for exponentials.

Aggregating these terms yields

$$\begin{aligned} \text{FLOPs}_d(k) &\approx 4dk^2 + 12k^2 + 2d\frac{k^2}{8} + 12\frac{k^2}{8} \\ &= \left(4d + 12 + \frac{d}{4} + \frac{3}{2}\right)k^2. \end{aligned}$$

Substituting $d = 16$ gives

$$\text{FLOPs}_{16}(k) \approx 81.5k^2,$$

which is on the order of 10^{11} FLOPs for $k = 32k$.

Bytes moved. To better capture implementation details, we count bytes per tile using the launch parameters that delivered the best runtime ($BLOCK_M = 64$, $BLOCK_N = 1024$). Each tile loads $64 \times d$ query values once ($4BLOCK_Md$ bytes), streams $1024 \times d$ training values ($4BLOCK_Nd$ bytes), and writes the partial PDF and weighted sums ($4BLOCK_M$ and $4BLOCK_Md$ bytes). All of this traffic goes to and from GDDR6, so for $d = 16$ a tile moves roughly

$$\begin{aligned} \text{Bytes}_{\text{tile}} &\approx 4(BLOCK_Md + BLOCK_Nd \\ &\quad + BLOCK_M + BLOCK_Md) \\ &= 4(2BLOCK_Md + BLOCK_Nd \\ &\quad + BLOCK_M) \\ &\approx 7.4 \times 10^4 \text{ bytes}. \end{aligned}$$

The full problem processes $(k/BLOCK_M) \times (k/BLOCK_N)$ such tiles, so the total GDDR6 traffic is

$$\begin{aligned} \text{Bytes}_{16}(k) &\approx \text{Bytes}_{\text{tile}} \times \frac{k}{BLOCK_M} \times \frac{k}{BLOCK_N} \\ &\approx 1.13k^2 \text{ bytes}. \end{aligned}$$

Arithmetic intensity. Dividing FLOPs by bytes gives

$$\begin{aligned} I_d(k) &= \frac{\text{FLOPs}_d(k)}{\text{Bytes}_d(k)} \\ &\approx \frac{(4d + 12 + \frac{d}{4} + \frac{3}{2})k^2}{4(\frac{9}{8}dk + \frac{k}{8})} \\ &\sim C(d)k \quad \text{for large } k, \end{aligned}$$

with

$$C(d) \approx \frac{4d + 12 + d/4 + 3/2}{4 \cdot (9d/8)} = \frac{(17/4)d + 27/2}{9d/2}.$$

For $d = 16$ this simplifies to

$$I_{16}(k) \approx \frac{\text{FLOPs}_{16}(k)}{\text{Bytes}_{16}(k)} \approx \frac{81.5k^2}{1.13k^2} \approx 72 \text{ flops/byte}.$$

This tile-aware estimate more closely reflects the measured arithmetic intensity. Comparing against the A6000 specs (Tensor Core peak of 155 TFLOP/s versus ≈ 770 GB/s of memory bandwidth), the machine balance is roughly 200 flops/byte. Since our kernel sustains over 70 flops/byte, it lies well into the compute-bound regime on the RTX A6000. Nsight Compute reports an empirical arithmetic intensity of roughly 95 FLOPs/byte for the score kernel on the A6000, which is broadly in line with the 72 FLOPs/byte predicted by the tile-level model above. Minor differences stem from additional bookkeeping work (atomics, reductions) and from Nsight’s finer-grained accounting of texture/cache traffic, but both figures agree that the kernel operates far above the bandwidth roofline. Because the kernel mixes Tensor-Core GEMMs with FP32 scalar work (norms, exponentials, atomics), the effective machine balance sits between the tensor-core roof (≈ 200 flops/byte) and the FP32 roof (≈ 50 flops/byte). The observed 70–95 flops/byte therefore straddle these two limits: the GEMM portion is partially bandwidth-limited relative to Tensor Cores, while the scalar portion is compute-limited relative to FP32 ALUs.

4. Results

We now summarize the empirical behavior of the 16-D implementation on the RTX A6000. For each n_{train} in $\{4k, 8k, 16k, 32k\}$ we set $n_{\text{test}} = n_{\text{train}}/8$, draw data from a simple 16-D Gaussian mixture, and run three baselines: scikit-learn KDE, Torch SD-KDE (GEMM-based), and Triton SD-KDE (Tensor-Core GEMMs for both KDE and score). Figure 1 shows that the Triton SD-KDE rapidly outpaces both baselines as n grows, while remaining numerically close to the Torch SD-KDE reference.

We also measure utilization by combining the flop model above with the measured runtimes. As shown in Figure 2, the Torch SD-KDE baseline reaches only a few percent of the A6000 Tensor Core peak, whereas the Triton implementation climbs into the multi-digit range once n_{train} exceeds 8k. This confirms that the 16-D Tensor-Core formulation is firmly compute-bound and that additional tuning effort should focus on kernel fusion and occupancy rather than memory traffic.

To compare against a specialized KDE implementation, we also benchmarked PyKeOps LazyTensor operators at $n_{\text{train}} = 32k$, $n_{\text{test}} = 4k$. PyKeOps currently represents the state of the art for large-scale kernel reductions, so we report both its Gaussian KDE runtime and its SD-KDE runtime as a reference point. Table 1 shows that Triton SD-KDE runs $1.4\times$ faster than the PyKeOps 16-D KDE baseline and $9.8\times$ faster than the PyKeOps 16-D SD-KDE implementation.

Method	Runtime (ms)	Rel to Triton
Triton 16-D SD-KDE	2.23	$1\times$
PyKeOps 16-D KDE	3.06	$1.4\times$
PyKeOps 16-D SD-KDE	21.73	$9.77\times$

Table 1. Runtime comparison at $n_{\text{train}} = 32k$, $n_{\text{test}} = 4k$. PyKeOps rows report Gaussian KDE and SD-KDE (LazyTensor) runtimes, while Triton executes the full SD-KDE pipeline.

4.1. Performance optimizations

Nsight Systems traces show that roughly 95% of the SD-KDE runtime at $n_{\text{train}} = 32k$, $n_{\text{test}} = 4k$ is spent computing the empirical score. We therefore concentrated optimization effort on the score kernel, sweeping the launch parameters to raise utilization. Specifically we varied:

- $BLOCK_M \in \{32, 64, 128, 256\}$,
- $BLOCK_N \in \{32, 64, 128, 256, 512, 1024\}$,
- $num_warps \in \{1, 2, 4, 8\}$,
- $num_stages \in \{1, 2, 4\}$,

and selected the combination that minimized runtime. For this workload we found that $BLOCK_M = 64$, $BLOCK_N = 1024$, $num_warps = 2$, and $num_stages = 2$ gave the best overall performance, increasing measured FLOP utilization by more than $2\times$ relative to smaller-tile settings. The same sweep can be repeated for different problem sizes if further tuning is needed.

Beyond the launch-parameter sweep, two design choices account for most of the speedup over a standard tiling implementation:

1. **Tensor-Core tiling:** all high-dimensional dot products are processed as 16×16 tiles via `tl.dot` with Tensor Core acceleration, allowing the GEMM portion of the score and KDE kernels to run near the TF32 roofline.
2. **Streaming accumulation:** we never materialize full $n_{\text{train}} \times n_{\text{train}}$ (or query) matrices; instead we stream tiles through registers and rely on atomic reductions to keep global-memory traffic linear in n .

Together these optimizations push the kernels close to the hardware limit. Nsight Compute’s “Speed of Light” report shows $\sim 68\%$ SM throughput and roughly 90% L1 throughput for the score kernel at $n_{\text{train}} = 32k$ and $n_{\text{test}} = 4k$, which is consistent with the tensor-core utilization trends in Figure 2: we cannot reach the nominal Tensor Core peak because the kernel still executes substantial non-Tensor-Core work (norms, exponentials, atomics). At these utilization levels we stopped further low-level tuning, since additional

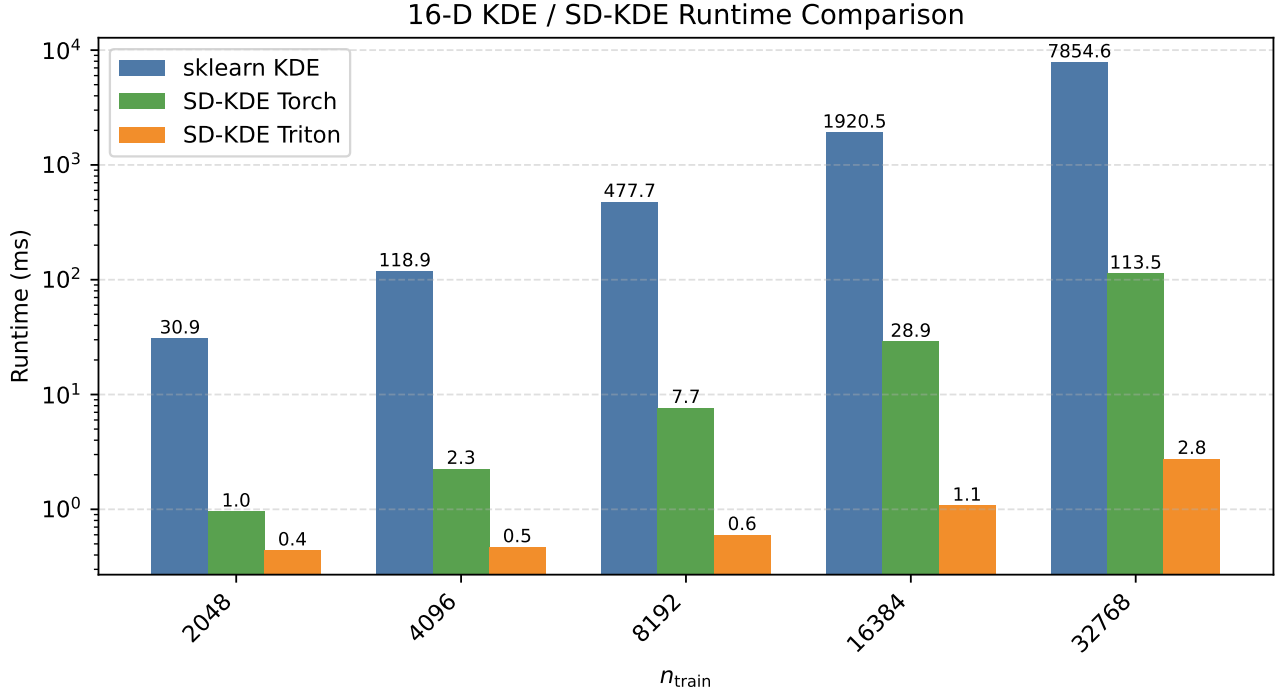


Figure 1. Runtime comparison for 16-D KDE/SD-KDE across n_{train} up to 32,768 ($n_{\text{test}} = n_{\text{train}}/8$).

optimizations would likely yield only modest incremental gains.

Impact Statement

This paper presents work whose goal is to accelerate SD-KDE and KDE kernels on GPUs. We expect the primary impact to be enabling practitioners to apply these estimators to larger datasets; we do not foresee unique societal consequences outside the usual considerations for density estimation applications.

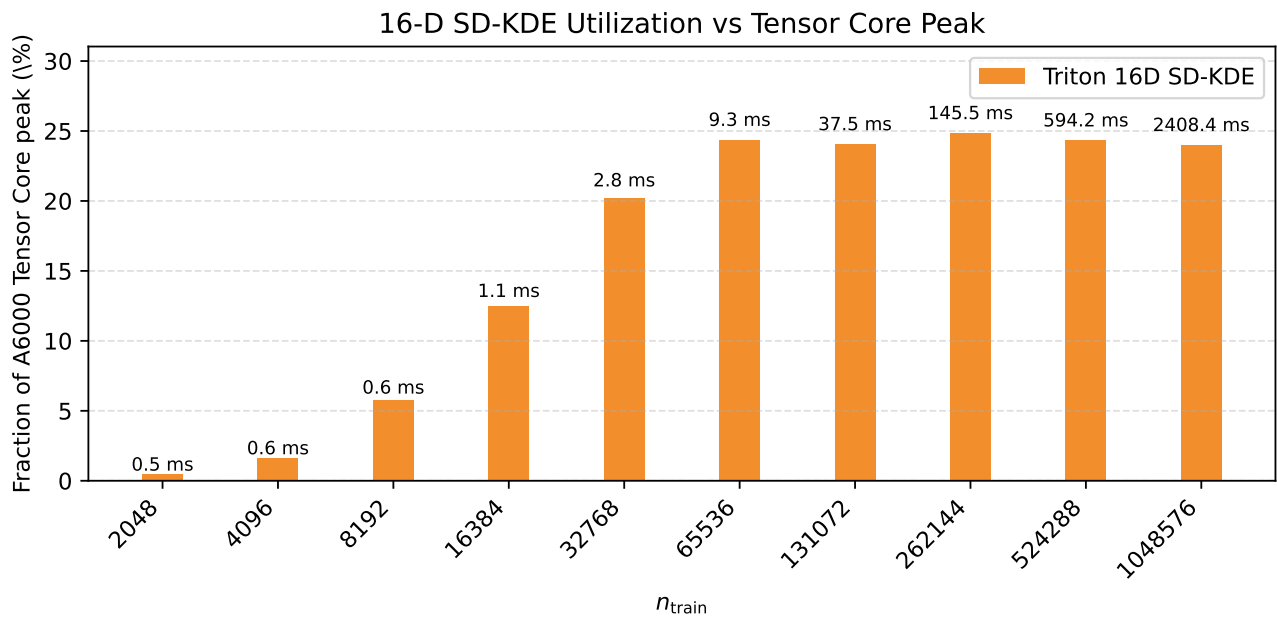


Figure 2. Utilization (percentage of RTX A6000 Tensor Core peak) for the 16-D SD-KDE pipeline, computed via the flop model from Section 3; bars are annotated with the observed runtimes (ms).

References

Epstein, E. L., Dwaraknath, R. V., Sornwanee, T., Winnicki, J., and Liu, J. W. SD-KDE: Score-debiased kernel density estimation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=fvho95EtPu>.